Longest Substring Program

Aim/Objective:

To develop a Java program to find the longest substring in a given string without repeating characters and display its length.

System Requirements:

• Operating System: Windows or any operating system supporting Java.

• Programming Language: Java.

• Java Development Kit (JDK).

• Text Editor or IDE: VS Code / Eclipse / IntelliJ IDEA.

• Command Prompt or Terminal.

Algorithm/Logic Description:

1. Start.
2. Read a string from the user.
3. Create an empty string named longest.
4. Use an outer loop to select every starting position of the string.
5. Create an empty string named current for each starting position.
6. Use an inner loop to read characters from the current position.
7. Check whether the current character already exists in current.
8. If the character is repeated, stop the inner loop.
9. Otherwise, add the character to current.
10. Compare the length of current with the length of longest.
11. If current is longer, store it in longest.
12. Repeat until all possible substrings are checked.
13. Display the length of the longest substring.
14. Display the longest substring.
15. Stop.

UML Class Diagram:

```
+-------------------------------------------+
|              LongestSubstring             |
+-------------------------------------------+
|                                           |
+-------------------------------------------+
| + findLongest(str : String) : String      |
| + main(args : String[]) : void            |
+-------------------------------------------+
                    |
                    | uses
                    v
             +-------------+
             |   Scanner   |
             +-------------+
             | nextLine()  |
             | close()     |
             +-------------+
```

Source Code:

```
import java.util.Scanner;

public class LongestSubstring {
    static String findLongest(String str) {
```

```
        String longest = "";

        for (int i = 0; i < str.length(); i++) {

            String current = "";

            for (int j = i; j < str.length(); j++) {

                char ch = str.charAt(j);

                // Check whether character is already present
                if (current.indexOf(ch) != -1) {
                    break;
                }

                current = current + ch;

                // Store the longest substring
                if (current.length() > longest.length()) {
                    longest = current;
                }
            }
        }

        return longest;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        String longest = findLongest(str);

        System.out.println("Length: " + longest.length());
        System.out.println("Longest substring: " + longest);

        sc.close();
    }
}
```

Compilation and Execution Commands:

```
Save the file as: LongestSubstring.java

Compile:
javac LongestSubstring.java

Run:
java LongestSubstring
```

Output Screenshot:

```
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> javac LongestSubstring.java
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> java LongestSubstring
Enter a string: abcabcbb
Length: 3
Longest substring: abc
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> javac LongestSubstring.java
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> java LongestSubstring
Enter a string: pwwkew
Length: 3
Longest substring: wke
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> javac LongestSubstring.java
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> java LongestSubstring
Enter a string: pwke
Length: 4
Longest substring: pwke
```

Result:

Thus, the Java program was successfully executed and the longest substring without repeating characters was found along with its length.